



Facultad de Ingeniería

**Fundamentos de Ingeniería de Software
020 - 86**

Ana Karina Roa Mora-20232020118

Juan Pablo Galindo Florez - 20231020230

Stevens Camilo Llanos Acero – 20231020221

Mayo 21, 2026

Manual del Programador – QueBoleta Backend

Descripción General

QueBoleta es una aplicación desarrollada con Spring Boot para la gestión de eventos, compra de entradas y control de acceso mediante códigos QR.

El sistema implementa:

- Autenticación JWT
- Roles de usuario
- CRUD de eventos
- Compra de entradas
- Generación y validación de QR
- Persistencia con MySQL
- Seguridad con Spring Security

Acceso al proyecto

Enlace al repositorio: <https://github.com/Juangf1222/QueBoleta.git>

Tecnologías utilizadas

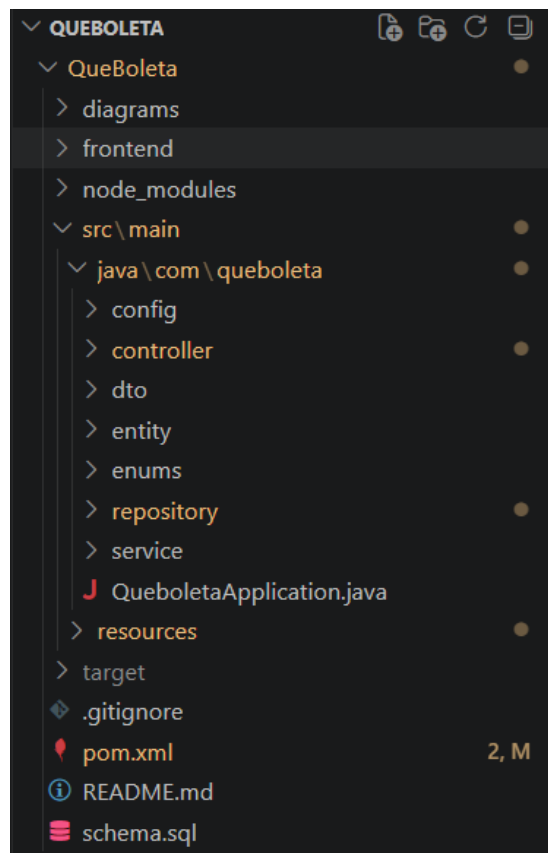
- Java 17+ : Lenguaje principal
- Spring Boot: Framework Backend.
- Spring Security: Seguridad y autenticación.
- JWT: Manejo de tokens.
- Spring Data JPA: Persistencia.
- Hibernate: ORM.
- MySQL: Base de datos.
- Maven: Gestión de dependencias.
- Swagger/OpenAPI: Documentación API:

Arquitectura del proyecto

El proyecto sigue una arquitectura por capas:

Controller → Service → Repository → Database

Y las clases se encuentran contenidas en paquetes:



Empaquetado de clases – Visual Studio Code

Diagramas – Implementación

➤ Casos de uso

- **Gestión de Eventos**

El caso de uso relacionado con los eventos fue implementado principalmente en las clases EventoController, EventoService y EventoRepository. El organizador o administrador puede crear eventos proporcionando información como nombre, artista, lugar, fecha, precio y cantidad de entradas disponibles. El sistema almacena estos datos en la entidad Evento, genera automáticamente un enlace único para el evento y controla la disponibilidad de entradas mediante el atributo disponibilidad. También se implementaron funcionalidades para editar eventos, buscar eventos por nombre, consultar eventos activos y cancelar eventos cambiando su estado a CANCELADO. Durante una compra, el sistema actualiza automáticamente el stock de entradas disponibles y genera los códigos QR correspondientes para cada entrada adquirida.

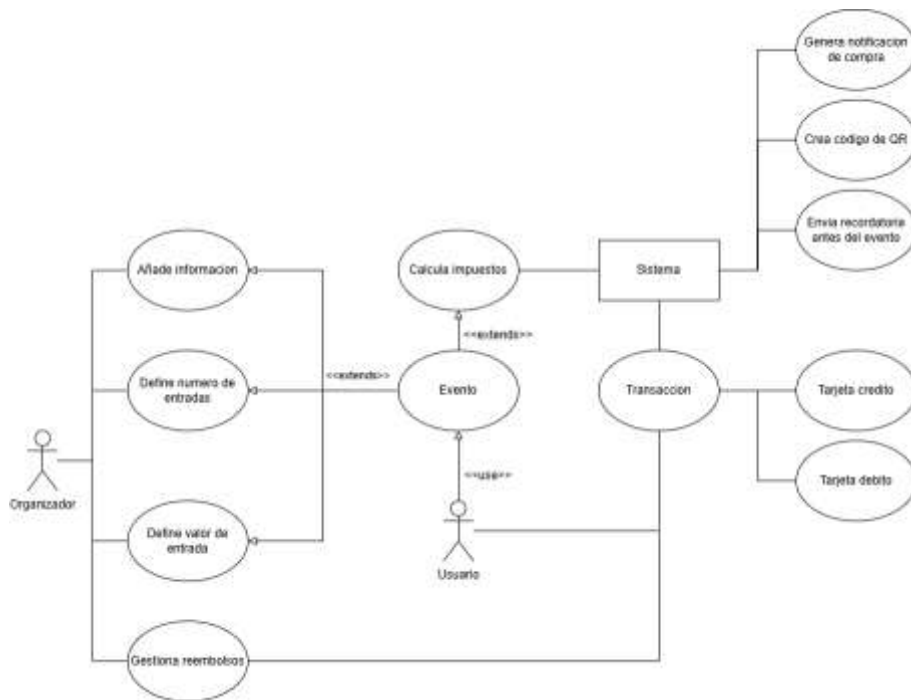


Diagrama de Casos de Uso – Gestión de Eventos

- **Inicio de sesión**

El caso de uso de inicio de sesión fue implementado utilizando Spring Security y autenticación JWT. El usuario puede registrarse mediante el endpoint /api/auth/registro, proporcionando nombre, correo, contraseña y rol. Posteriormente puede iniciar sesión usando /api/auth/login. El sistema valida las credenciales mediante AuthenticationManager y CustomUserDetailsService, que consulta los usuarios almacenados en la base de datos. Una vez autenticado, se genera un token JWT usando la clase JwtUtils, el cual contiene la información del usuario y su rol. Este token debe enviarse en cada petición protegida mediante el encabezado Authorization: Bearer TOKEN. El filtro JwtAuthFilter intercepta las solicitudes, valida el token y autoriza el acceso según el rol del usuario.

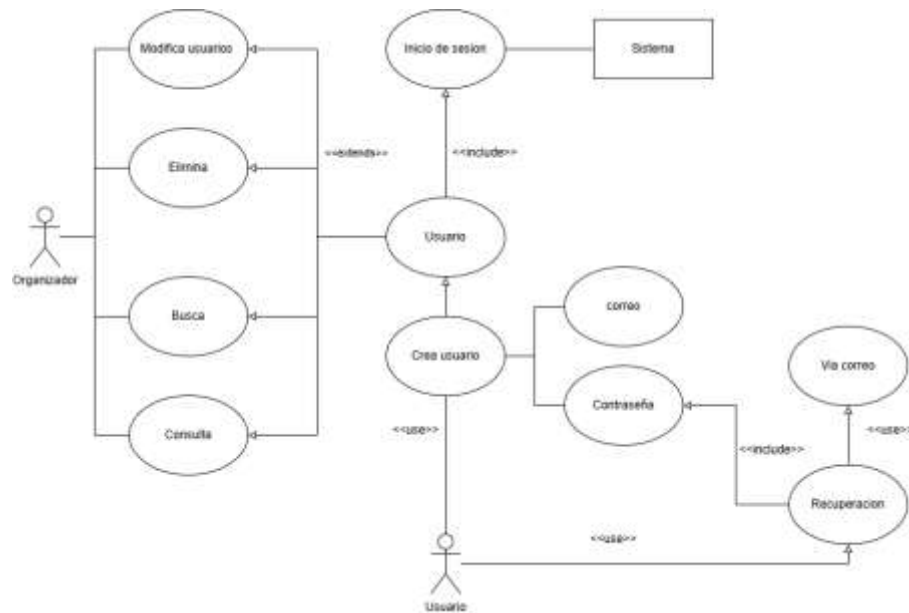
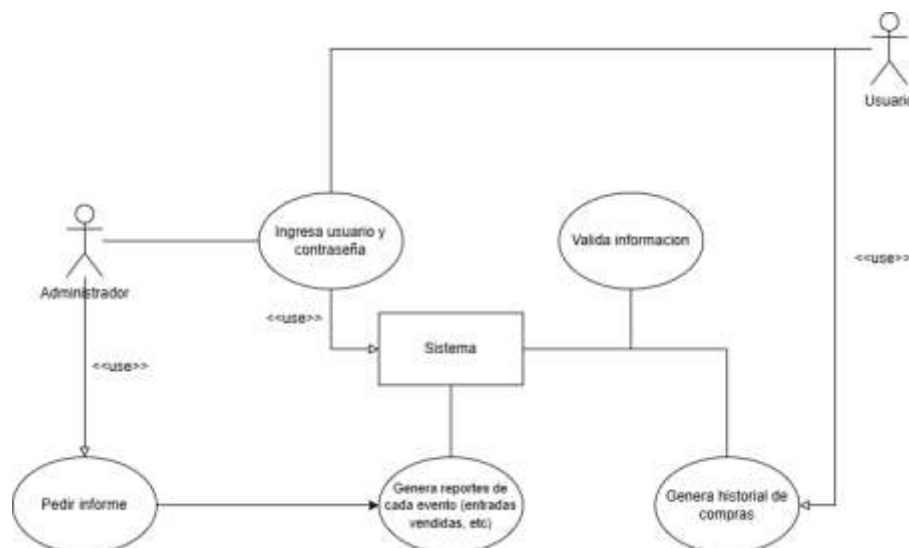


Diagrama de Casos de Uso – Inicio de Sesión

- **Reportes**

El caso de uso de reportes fue implementado mediante la entidad Reporte y las relaciones existentes con los eventos y transacciones. El sistema puede almacenar información relacionada con entradas vendidas e ingresos generados por cada evento. Aunque actualmente la generación de reportes se encuentra parcialmente implementada, la estructura ya permite registrar estadísticas de ventas e ingresos totales. La clase Administrador contiene métodos como verReportes() y agregarReporte(), lo que permite asociar reportes a los administradores. Además, las transacciones almacenadas en la tabla transaccion sirven como base para calcular datos como cantidad de boletos vendidos, ingresos totales y estado de las compras, lo que facilita una futura expansión del módulo de reportes y análisis del sistema.



➤ **Diagrama de contexto**

El diagrama de contexto se implementó mediante una arquitectura cliente-servidor basada en API REST. El “Sistema de gestión de eventos” corresponde a la aplicación Spring Boot desarrollada en el backend. Los actores externos del diagrama, como el usuario, el administrador, el organizador y el sistema de pagos, interactúan con el sistema mediante endpoints REST definidos en los controladores (AuthController, EventoController, CompraController, entre otros). El usuario puede consultar eventos y realizar compras; el administrador puede crear, editar o cancelar eventos; y el sistema de pagos fue representado mediante el servicio PagoService, encargado de manejar la lógica de procesamiento de pagos. Toda la comunicación se realiza mediante solicitudes HTTP y respuestas JSON, utilizando autenticación JWT para proteger las operaciones privadas del sistema.

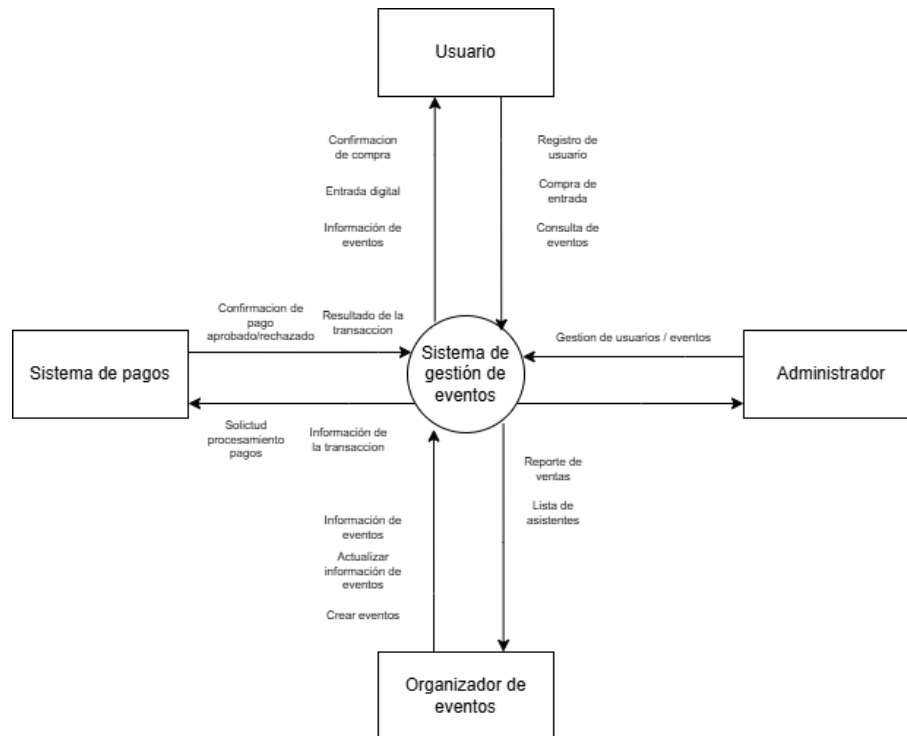


Diagrama de Contexto

➤ **Diagrama de clases**

El diagrama de clases fue implementado en el proyecto utilizando entidades Java con JPA y Spring Boot. La clase abstracta Usuario funciona como clase base y utiliza herencia SINGLE_TABLE, permitiendo que Cliente, Administrador y PersonalAcceso compartan la misma tabla en la base de datos, diferenciándose mediante el atributo rol. Las relaciones mostradas en el diagrama también fueron implementadas mediante anotaciones JPA como

@OneToMany y @ManyToOne; por ejemplo, un Cliente puede tener múltiples Entrada, mientras que una Transaccion se relaciona con un Usuario y un Evento. Los métodos planteados en el modelo UML, como generarQR(), consultarDisponibilidad(), realizarCompra() o solicitarReembolso(), fueron desarrollados dentro de las entidades y servicios correspondientes. Además, los enumeradores del diagrama (EstadoEvento, EstadoEntrada, MetodoPago y TipoNotif) fueron implementados como enums de Java para controlar estados y tipos dentro de la aplicación.

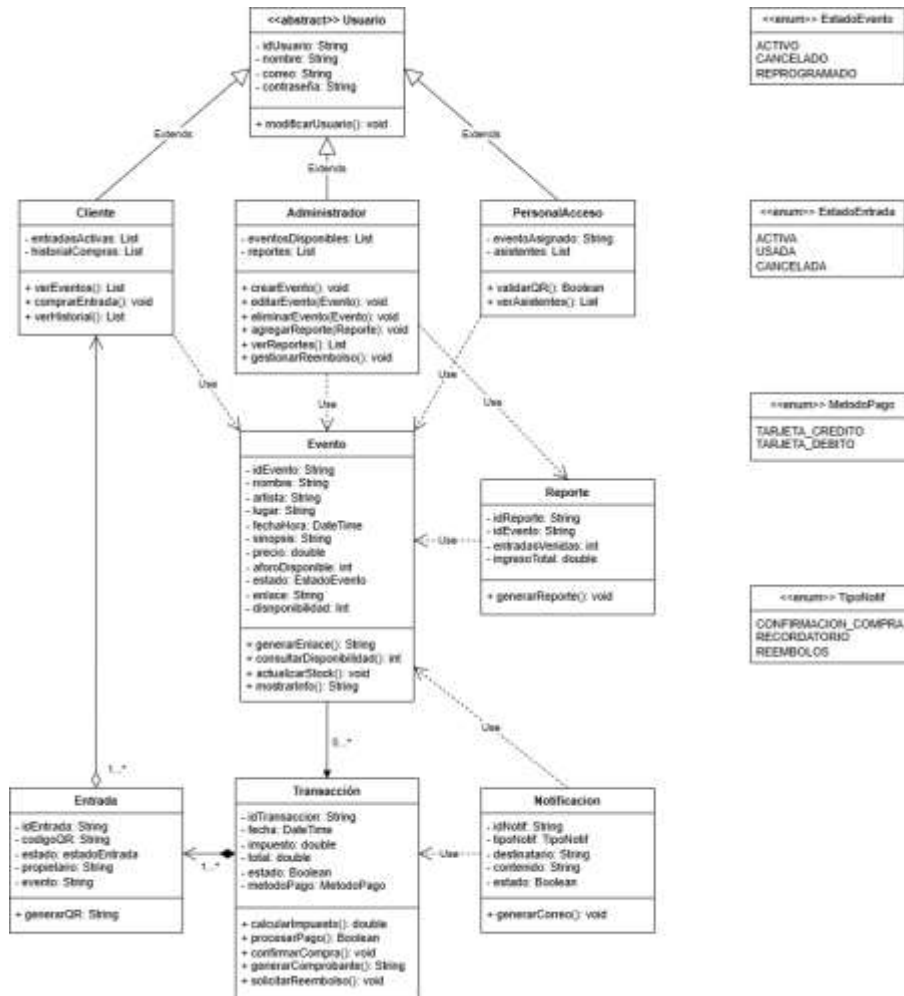


Diagrama de Clases

➤ Diagrama de secuencia

El diagrama de secuencia muestra el flujo completo de compra a través de cinco participantes: Cliente, Página Web, Sistema de Ventas, Punto de Pago y Banco. En el proyecto, ese flujo quedó implementado de la siguiente manera: el Cliente envía su solicitud a través del endpoint POST /api/compras del CompraController, que representa la interacción con la Página Web; este delega en CompraService.realizarCompra(), que actúa como el Sistema de Ventas, verificando disponibilidad del evento y creando la Transaccion. Desde allí se simula la comunicación con el

Punto de Pago y el Banco a través de PagoService, que valida los datos de tarjeta del CompraRequest y retorna un resultado de aprobación, tras lo cual CompraService confirma la compra, descuenta el stock del evento, genera las Entradas con su código QR mediante entrada.generarQR(), y dispara el NotificacionService para enviar la boleta digital al correo del cliente, devolviendo finalmente un CompraResponse con los códigos QR, el resumen financiero y el mensaje de confirmación.

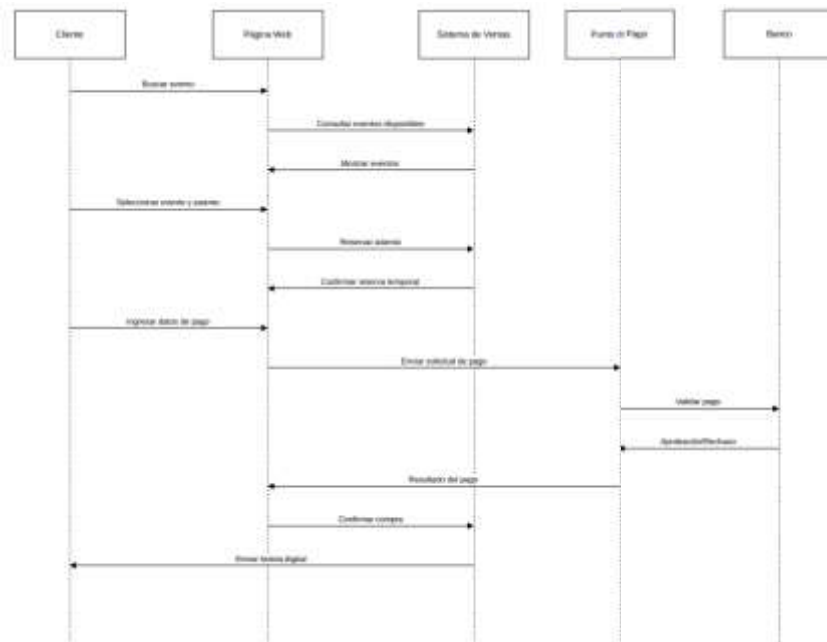


Diagrama de Secuencia

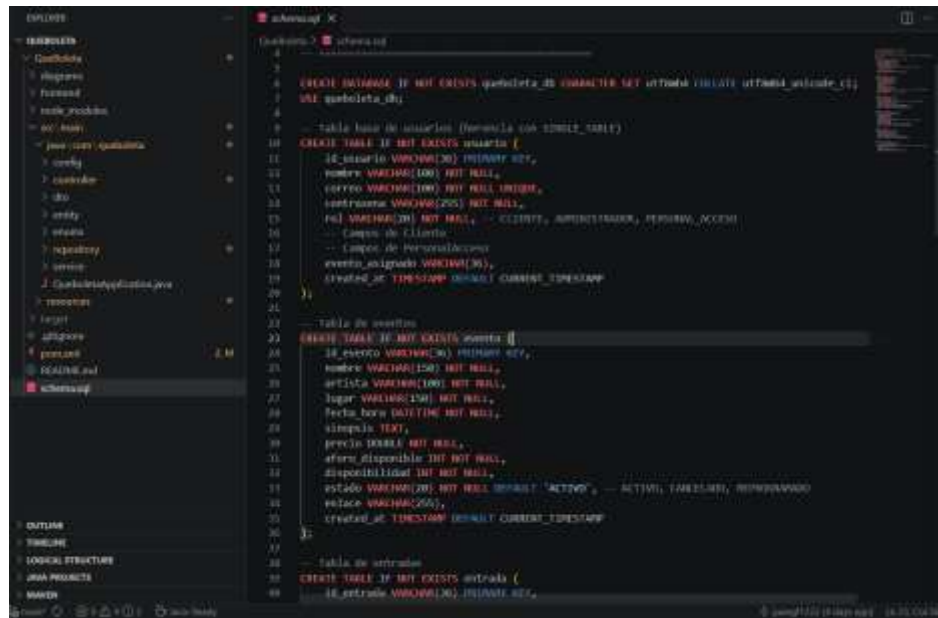
Configuración del entorno

Se requiere:

- Java JDK 17 o superior
- Maven
- MySQL Server y MySQL Workbench
- NetBeans o Visual Studio Code
- Postman

Configuración de la base de datos

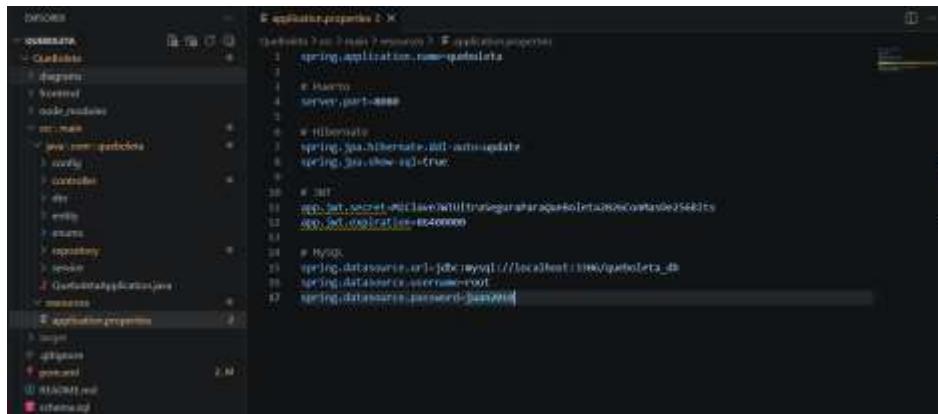
Crear la base de datos ejecutando el script SQL (archivo schema.sql del proyecto):



```
1
2
3
4 CREATE DATABASE IF NOT EXISTS quiboleta_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
5 USE quiboleta_db;
6
7 -- tabla base de usuarios (funciona con double table)
8
9 CREATE TABLE IF NOT EXISTS usuarios (
10     id_usuario VARCHAR(30) PRIMARY KEY,
11     nombre VARCHAR(100) NOT NULL,
12     correo VARCHAR(100) NOT NULL UNIQUE,
13     contrasena VARCHAR(255) NOT NULL,
14     rol VARCHAR(20) NOT NULL, -- CLIENTE, ADMINISTRADOR, RESEÑA, ACCESO
15     -- Campos de cliente
16     -- Campos de personalizacion
17     eventos_asignados VARCHAR(50),
18     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
19 );
20
21 -- tabla de eventos
22
23 CREATE TABLE IF NOT EXISTS eventos (
24     id_evento VARCHAR(30) PRIMARY KEY,
25     nombre VARCHAR(150) NOT NULL,
26     artista VARCHAR(100) NOT NULL,
27     lugar VARCHAR(150) NOT NULL,
28     fecha_hora DATETIME NOT NULL,
29     sinopsis TEXT,
30     precio DOUBLE NOT NULL,
31     afere_disponible INT NOT NULL,
32     disponibilidad INT NOT NULL,
33     estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO', -- ACTIVO, FALLECIDO, REMOVIDO
34     enlace VARCHAR(255),
35     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
36 );
37
38 -- tabla de entradas
39
40 CREATE TABLE IF NOT EXISTS entradas (
41     id_entrada VARCHAR(30) PRIMARY KEY,
```

Archivo schema.sql – Visual Studio Code

Configuración application.properties:



```
1
2 spring.application.name=quiboleta
3
4 # Puerto
5 server.port=8080
6
7 # Hibernate
8 spring.jpa.hibernate.ddl-auto=update
9 spring.jpa.show-sql=true
10
11 # JWT
12 app.jwt.secret=${JWT_SECRET_KEY}
13 app.jwt.expiration=${JWT_EXPIRATION}
14
15 # Logs
16 spring.datasource.url=jdbc:mysql://localhost:3306/quiboleta_db
17 spring.datasource.username=root
18 spring.datasource.password=quiboleta
```

Archivo application.properties – Visual Studio Code

Ejecución del Proyecto

Para el Backend:

- Abrir proyecto Maven.
- Esperar a que se descarguen las dependencias.
- Ejecutar: En la clase main, Click derecho → run.

Para el Frontend:

Desde la terminal:



```
PS C:\Users\camil\Documents\NetBeansProjects\Queboleta> cd Queboleta
PS C:\Users\camil\Documents\NetBeansProjects\Queboleta\Queboleta> cd frontend
PS C:\Users\camil\Documents\NetBeansProjects\Queboleta\Queboleta\frontend> npm run dev

> frontend@0.0.0 dev
> vite

VITE v4.0.13 ready in 500 ms
➔ Local:   http://localhost:3173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

Terminal – Visual Studio Code

Seguridad del sistema

La autenticación se realiza mediante JSON Web Tokens:

Clase principal: config/JwtUtils.java.

Funciones:

- Generar token
- Validar token
- Extraer correo
- Verificar expiración

Filtro JWT: config/JwtAuthFilter.java

Responsabilidades:

- Interceptar requests
- Leer Authorization Header
- Validar token

- Cargar usuario autenticado

Spring Security: config/SecurityConfig.java

Configuraciones:

- CSRF deshabilitado
- Sesiones stateless
- Rutas públicas
- Protección JWT

Roles del sistema

Cliente: Puede:

- Registrarse
- Iniciar sesión
- Comprar entradas
- Ver sus entradas

Administrador: Puede:

- Crear eventos
- Editar eventos
- Cancelar eventos
- Ver todos los eventos

Personal_acceso: Puede:

- Validar entradas QR

Modelo de datos

Usuario: entity/Usuario.java

Clase abstracta base. Heredan: Cliente, Administrador, Personal_Acceso.

Evento: entity/Evento.java

Representa conciertos o eventos disponibles. Atributos:

- nombre
- artista
- fechaHora
- precio
- disponibilidad
- estado

Entrada: entity/Entrada.java

Representa una entrada comprada. Atributos:

- codigoQR
- estado
- evento
- propietario

Transaccion: entity/Transaccion.java

Registra compras realizadas. Atributos:

- usuario
- evento
- cantidadBoletos
- valorTotal
- estado

Flujo de Compra

1. Paso 1 → Login.
2. Paso 2 → Comprar entrada.
3. Paso 3 → Generación de entradas.
4. Paso 4 → Validación QR.

Controladores

AuthController: controller/AuthController.java

Maneja:

- Registro
- Login

EventoController: controller/EventoController.java

CRUD de eventos.

CompraController: controller/CompraController.java

Maneja compras.

EntradaController: controller/EntradaController.java

Maneja entradas del usuario.

AccesoController: controller/AccesoController.java

Valida QR.

Servicios

UsuarioService: service/UsuarioService.java

Funciones:

- Registro
- Login
- Generación JWT

EventoService: service/EventoService.java

Funciones:

- Crear evento
- Editar evento

- Cancelar evento
- Buscar eventos

CompraService: service/CompraService.java

Funciones:

- Validar disponibilidad
- Crear transacción
- Generar entradas

EntradaService: service/EntradaService.java

Funciones:

- Generar QR
- Validar QR
- Cambiar estado de entrada

Manejo de estados

EstadoEvento: enums/EstadoEvento.java

- ACTIVO
- CANCELADO
- REPROGRAMADO

EstadoEntrada: enums/EstadoEntrada.java

- ACTIVA
- USADA
- CANCELADA

EstadoTransaccion: enums/EstadoTransaccion.java

- COMPLETADA
- REEMBOLSADA
- CANCELADA

Documentación Swagger

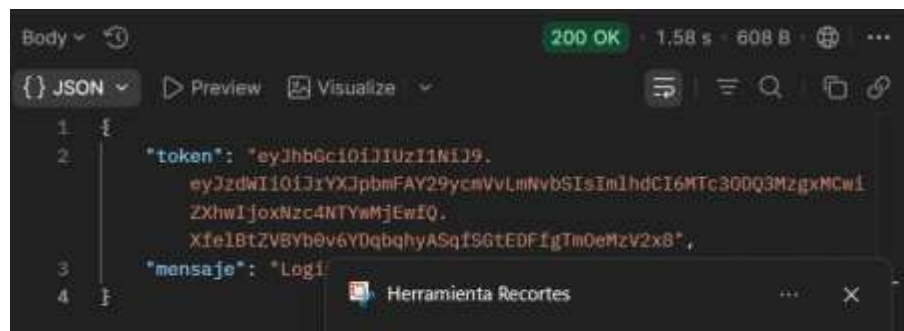
Disponible en: <http://localhost:8080/swagger-ui/index.html>

Reporte técnico de calidad

A continuación, se detallan las pruebas funcionales, de seguridad y de carga realizadas al Backend del sistema QueBoleta, desarrollado en Java con Spring Boot y MySQL. Se valida el cumplimiento de los requerimientos de negocio, la protección de rutas mediante JWT y la estabilidad del servidor.

1. Pruebas Funcionales:

- *Registro e inicio de sesión:*



- *Creación de evento:*

```
Body v ⓘ 201 Created • 1.54 s • 753 B • ...
{ } JSON v ▶ Preview 🖼 Visualize v
1 {
2   "idEvento": "fe1c5c79-7560-47e8-8f8c-c4b1b456706e",
3   "nombre": "Concierto VIP Feid",
4   "artista": "Feid",
5   "lugar": "Movistar Arena",
6   "fechaHora": "2026-12-01T20:00:00",
7   "sinopsis": null,
8   "precio": 300000.0,
9   "aforoDisponible": 5,
10  "disponibilidad": 5,
11  "estado": "ACTIVO",
12  "enlace": "https://queboleta.com/evento/fe1c5c79-7560-47e8-8f8c-c4b1b456706e"
13 }
```

- *Compra exitosa:*

```
Body v ⓘ 200 OK • 1.28 s • 1.02 KB • ...
{ } JSON v ▶ Preview 🖼 Visualize v
1 {
2   "idEvento": "fe1c5c79-7560-47e8-8f8c-c4b1b456706e",
3   "nombre": "Concierto VIP Feid",
4   "artista": "Feid",
5   "lugar": "Movistar Arena",
6   "fechaHora": "2026-12-01T20:00:00",
7   "sinopsis": null,
8   "precio": 300000.0,
9   "aforoDisponible": 5,
10  "disponibilidad": 5,
11  "estado": "ACTIVO",
12  "enlace": "https://queboleta.com/evento/fe1c5c79-7560-47e8-8f8c-c4b1b456706e"
13 }
```

- *Validación de Stock:*

```
Body v ⓘ 500 Internal Server Error • 365 ms • 525 B • ...
{ } JSON v ▶ Preview 🖼 Visualize v ▶ Debug with AI v
1 {
2   "timestamp": "2026-05-11T04:37:00.009+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/api/compras"
6 }
```

2. Pruebas de Seguridad

- *Bloqueo a intrusos:*



A screenshot of a REST client interface. At the top, a status bar indicates a "403 Forbidden" error, with a response time of "19 ms" and a size of "541 B". Below this, the "Body" tab is selected, showing a JSON response. The JSON object contains the following fields: "timestamp" (a date string), "status" (403), "error" ("Forbidden"), and "path" (the API endpoint). The interface includes a "Preview" button and a link to "Try with different auth credentials".

```
1 {
2   "timestamp": "2026-05-11T04:39:43.585+00:00",
3   "status": 403,
4   "error": "Forbidden",
5   "path": "/api/reportes/dashboard"
6 }
```

- *Bloqueo de Roles:*



A screenshot of a REST client interface. At the top, a status bar indicates a "405 Method Not Allowed" error, with a response time of "104 ms" and a size of "567 B". Below this, the "Body" tab is selected, showing a JSON response. The JSON object contains the following fields: "timestamp" (a date string), "status" (405), "error" ("Method Not Allowed"), and "path" (the API endpoint). The interface includes a "Preview" button and a link to "Debug with AI".

```
1 {
2   "timestamp": "2026-05-11T04:41:35.624+00:00",
3   "status": 405,
4   "error": "Method Not Allowed",
5   "path": "/api/auth/registro"
6 }
```